

CODECHECK certificate 2026-022

Contrail Avoidance Non-CO2 Evaluation (CANE)



Item	Value
Title	Contrail Avoidance Non-CO2 Evaluation (CANE)
Authors	Jakob Smretschmig (0009-0003-6446-3039)
Ref. paper	Can we achieve significant climate mitigation by optimising only for contrails? https://doi.org/10.5194/jecats-2026-13
Repository	https://gitlab.tudelft.nl/jsmretschmig/jecats-paper-climate-mitigation https://doi.org/10.4121/cbdafa47-709d-45da-86e1-7fed28ec3582
Codecheckers	Heather Andrews (0000-0002-6637-2830)
Date of check	2026-08-13
Summary	All figures of the related publication were reproduced, generated by Jupyter notebooks provided in the repository. In order to reproduce the figures, two datasets are necessary. One dataset has been published by the author. The other dataset is already public data. Regarding the code, the notebooks require an extension that is available upon request (not distributed within the code repository) and a visualization code already open-source available via GitHub.
Codecheck repository	https://github.com/codecheckers/jecats-paper-climate-mitigation

Table 1: CODECHECK summary

Summary

Output	Comment
./figures/[CODECHECK]fig01.png	manuscript Figure 1
./figures/fig02.png	manuscript Figure 2
./figures/fig03.png	manuscript Figure 3
./figures/fig04.png	manuscript Figure 4
./figures/fig05.png	manuscript Figure 5
./figures/fig06_day[6, 7]_['ENOB', 'ENOR', 'EKDK', 'ESAA', 'EFIN', 'EETT', 'EVRR'].png	manuscript Figure 6
./figures/fig07.png	manuscript Figure 7
./figures/fig08.png	manuscript Figure 8
./figures/fig09.png	manuscript Figure 9
./figures/fig10.png	manuscript Figure 10
./figures/fig11.png	manuscript Figure 11
./figures/figC1.png	manuscript Figure C1
./figures/figC2.png	manuscript Figure C2
./figures/figC3.png	manuscript Figure C3
./figures/figD1.png	manuscript Figure D1
./figures/figE1.png	manuscript Figure E1

Table 2: Summary of output files generated

CODECHECKER notes

- The code repository has been reviewed by the codechecker, not only in terms of reproducibility of outputs but also other aspects such as repository structure, documentation and licensing information. All feedback was well received and has been processed by the author of the code repository.
- The code has been released via the Technische Universiteit Delft (TU Delft) GitLab with a DOI from the 4TU.ResearchData archive. The DOI of the code repository is <https://doi.org/10.4121/cbdafa47-709d-45da-86e1-7fed28ec3582>.
- The code repository contains Jupyter notebooks that generate the figures of the publication using data from 2 different sources. One dataset has been published by the author via the 4TU.ResearchData archive. The DOI of the data repository is <https://doi.org/10.4121/16ec2954-a493-41c8-b3f9-f17e9331c46b>. The other dataset corresponds to ERA5 tropopause data that is already published (<https://doi.org/10.26165/JUELICH-DATA/UBNGI2>). Each file of this last dataset is ~1.4 GB in size. The documentation of the notebook that uses files from this dataset (`./notebooks/figure-06.ipynb`) specifies which files must be downloaded and how they should be renamed.
- In order to reproduce the figures, `pycontrails-bada 0.7.7` and ``sankey.py`` are required. Both are not distributed in the code repository:
 - o `pycontrails-bada` is a private extension to the open-source `pycontrails` module. The extension is accessible upon request. Instructions on how to request access are specified in the ``README.md``. In order to perform the CODECHECK, the codechecker got access to `pycontrails-bada 0.7.7`.
 - Note: the repository's ``pyproject.toml`` specifies ``pycontrails[complete]>=0.54.11``. During the CODECHECK, this dependency was resolved to `pycontrails 0.63.3`. From `pycontrails 0.63.3` changelog it is mentioned that ``DEFAULT_LOAD_FACTOR`` was removed in `pycontrails 0.63.3`. However `pycontrails-bada 0.7.7` still imports ``DEFAULT_LOAD_FACTOR`` from `pycontrails.core.aircraft_performance`. As a result, running `pycontrails-bada 0.7.7` with `pycontrails 0.63.3` produces an import error. To solve this the codechecker had to comment out line 22 in ``.venv/Lib/site-packages/pycontrails_bada/bada_model.py`` which imports ``DEFAULT_LOAD_FACTOR``.
 - o ``sankey.py`` is a Python module that is open-source and distributed via `pySankey` GitHub repository (<https://github.com/anazalea/pySankey>). The author refers to this repository and the edits that must be made to ``sankey.py`` in order to reproduce Figure 9 of the related paper.

The repository contains 17 Jupyter notebooks, from which 16 reproduce the figures of the related publication. One Jupyter notebook corresponds to a pipeline users can use and adapt to perform the same processing pipeline on a high-performance-computer (HPC) with their

own datasets. This CODECHECK refers only to the notebooks reproducing the figures of the related publication (incl. the appendixes).

The CODECHECK was performed in Windows 11 Enterprise (OS: Windows 11 Enterprise 24H2 (x86_64)) and Git Bash (git version 2.49.0.windows.1), within VSCode (version 1.128.0 user setup) using python 3.11.15 and the following Jupyter-related packages:

IPython : 9.16.1
ipykernel : 7.3.0
ipywidgets : 8.1.8
jupyter_client : 8.9.1
jupyter_core : 5.9.1
jupyter_server : 2.20.0
jupyterlab : 4.6.3
nbclient : 0.11.0
nbconvert : 7.17.1
nbformat : 5.11.0
notebook : 7.6.2
qtconsole : not installed
traitlets : 5.16.1

All notebooks were run with CPU Processor: Intel(R) Core(TM) i7-10610U @ 1.80 GHz.

Regarding environment setup, the author provides detailed instructions for reproducing the figures, including a list of dependencies and guidance on how to install them (the repository provides a `pyproject.toml`). The codechecker followed the respective instructions, including the setup of the recommended directory structure.

The codechecker downloaded the datasets mentioned in the `README.md`, placed the downloaded datasets in a `./data/` directory and created a `./figures/` folder.

Consequently, no modifications to the file paths in the notebooks were necessary. All figures generated during the codechecking procedure are stored in `./figures/`.

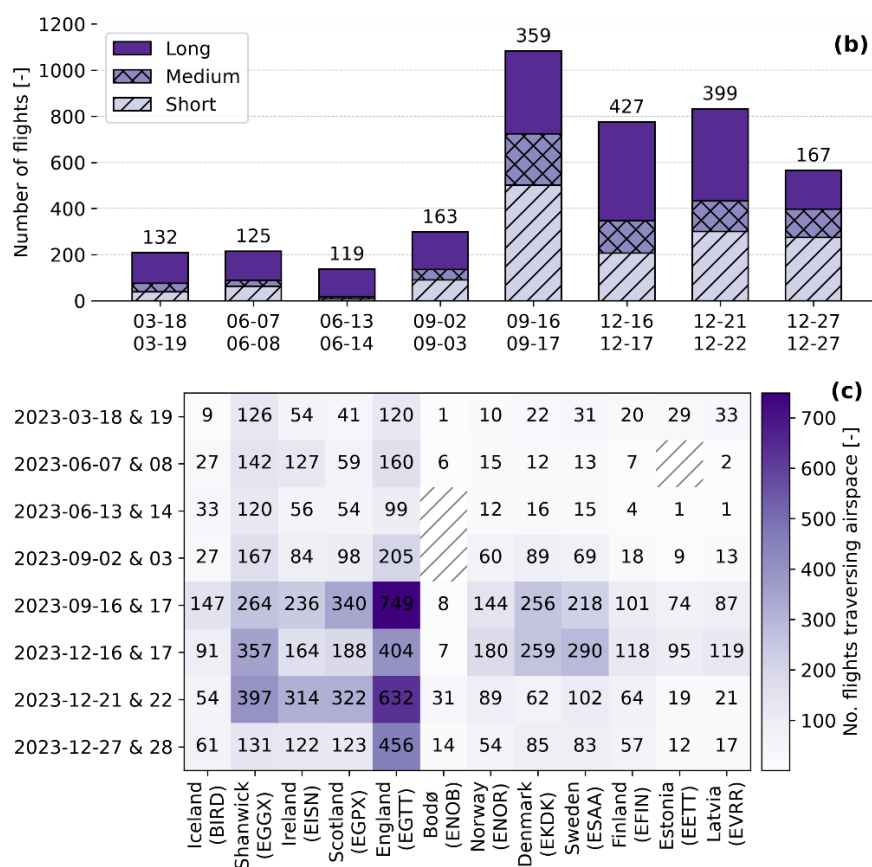
Manifest Files

Figure 1:

The figure is created by the ``./notebooks/figure-01.ipynb``. When running the notebook two issues appeared:

- 1) The notebook displayed and printed only panels b and c. Not panel a.
- 2) The numbers displayed at the top of the bars of panel b were different than in the related publication. It is important to note that the bar plot itself is identical to the one shown in the related publication; only the numerical labels above the bars differ.

This is the ``fig01.png`` created by the notebook ``./notebooks/figure-01.ipynb``:



After examining the generated figure and the notebook, the problem regarding point 1) is that ``bbox_inches="tight"`` does not work well with CartoPy map projections. When matplotlib calculates the "tight" bounding box, it does not account for the projected coordinate system properly. To fix this: the ``bbox_inches="tight"`` option in the ``plt.savefig`` instructions of ``plot_merged()`` and ``plot_merged_three()`` were changed to:

```
`plt.savefig(save_path, dpi=300, bbox_inches=None, pad_inches=0.1)`
```

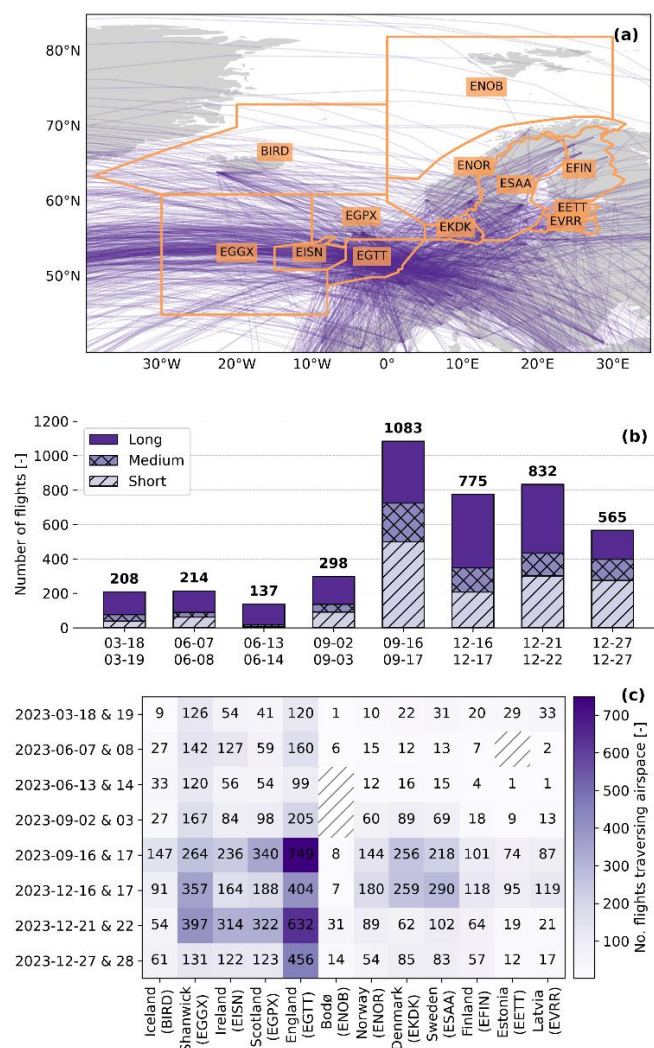
The ``pad_inches`` was specified to add a small margin around the figure.

The fix is tagged with inline comments “CODECHECK MODIFICATION” (see the notebook `\./notebooks/[CODECHECK]figure-01.ipynb``).

Regarding point 2), the plot itself is not the problem. Only the numerical labels printed on top of the bars. The problem is that ``ax2.bar_label(bars, ...)`` is only labelling the last set of bars (the last iteration of the loop). Thus, instead of labelling all bars with their total, it only labels the last ``haul_type``.

In order to fix this, instructions to print the sum were added to both ``plot_merged()`` and ``plot_merged_three()`` functions. Just like for point 1), this fix is tagged with inline comment entitled “CODECHECK MODIFICATION” (see the notebook `\./notebooks/[CODECHECK]figure-01.ipynb``).

The figure with both fixes (generated by `\./notebooks/[CODECHECK]figure-01.ipynb``) is the following and it is saved to `\./figures/[CODECHECK]fig01.png``:



Summary:

- point 1) is not an issue with the calculations: the issue is with CartoPy's interaction with matplotlib.
- Point 2) resulted from a mistake in the logic of the instructions, and affected only the presentation of the figure (i.e. its graphical display), and not to its content (the values of the bar plots).

Hence Figure 1 is reproduced.

Figure 2:

The figure is created by ``./notebooks/figure-02.ipynb`` and saved to ``./figures/fig02.png``.
The figure is reproduced.

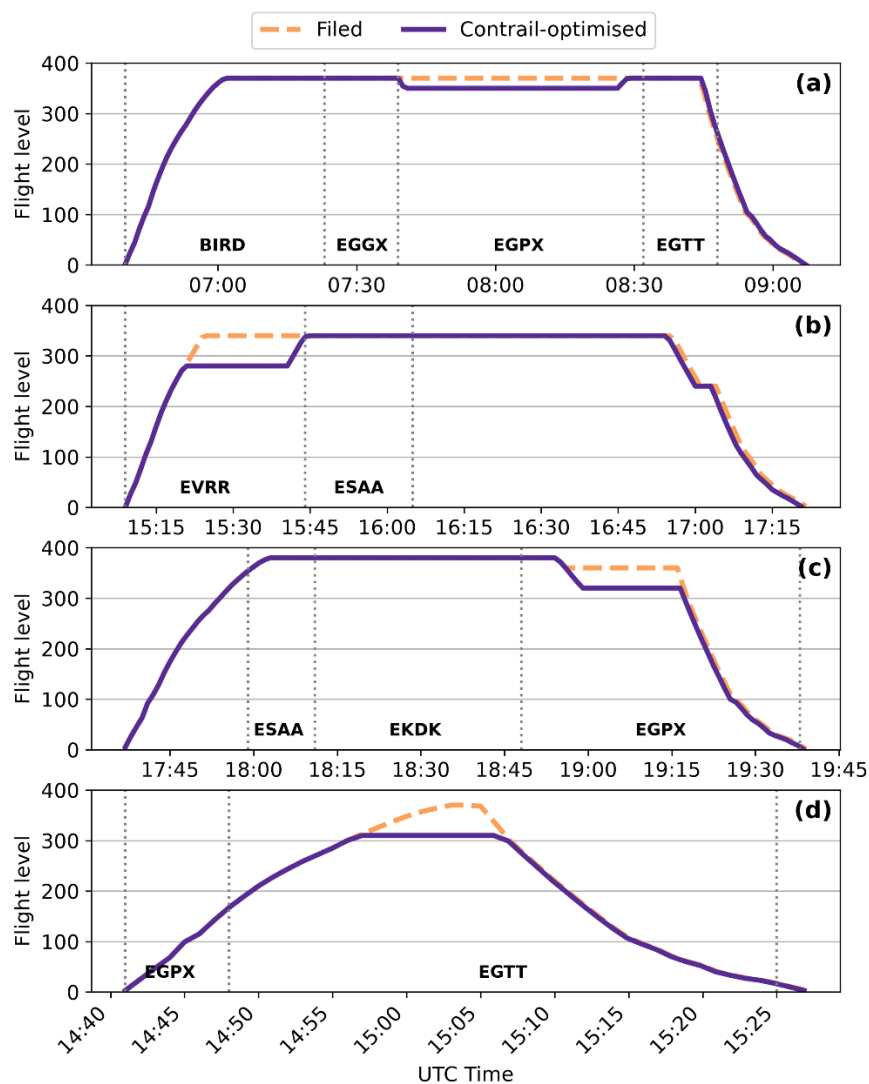


Figure 3:

The figure is created by ``./notebooks/figure-03.ipynb`` and saved to ``./figures/fig03.png``.
The figure is reproduced.

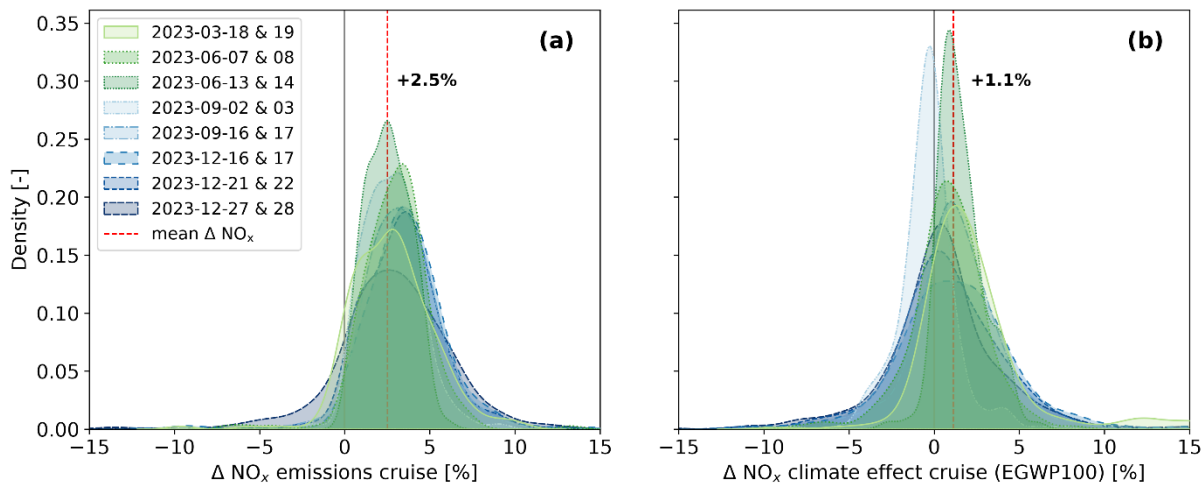


Figure 4:

The figure is created by ``./notebooks/figure-04.ipynb`` and saved to ``./figures/fig04.png``.
The figure is reproduced.

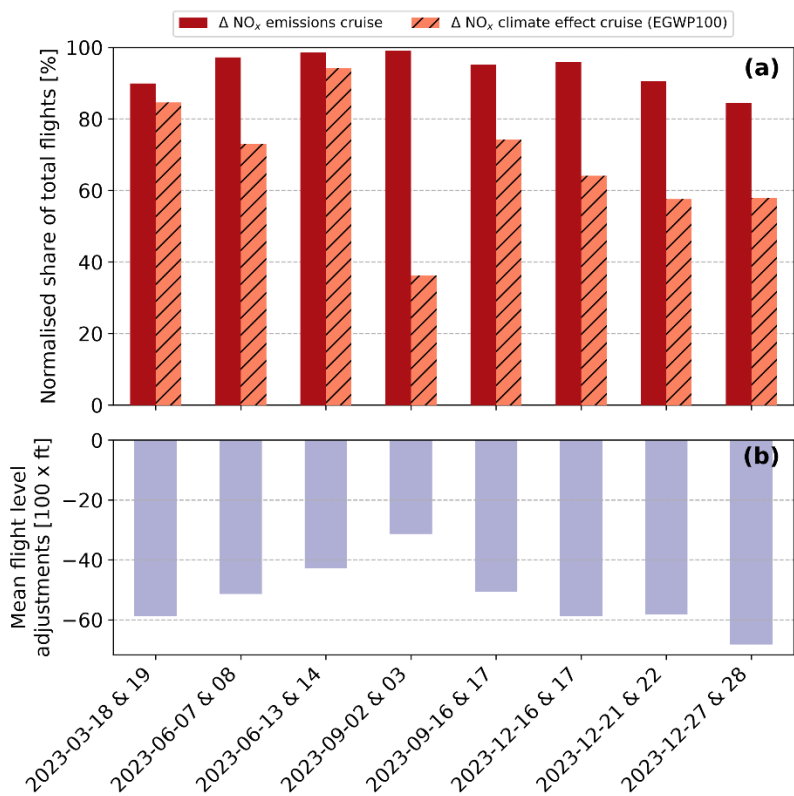


Figure 5:

The figure is created by `./notebooks/figure-05.ipynb` and saved to `./figures/fig05.png`.
The figure is reproduced.

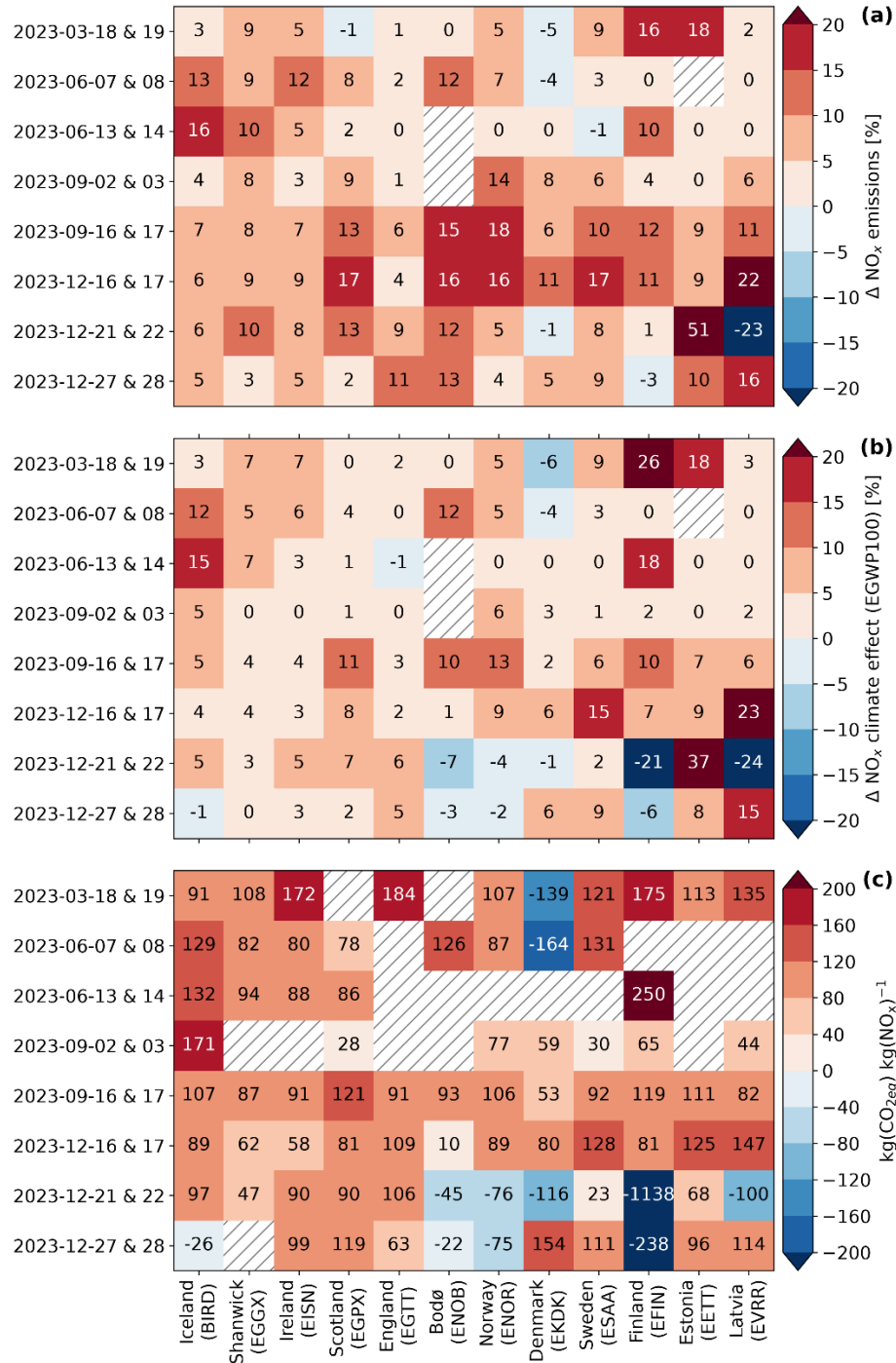


Figure 6:

The figure is created by `./notebooks/figure-06.ipynb` and saved to `./figures/ fig06\_day[6, 7]\_['ENOB', 'ENOR', 'EKDK', 'ESAA', 'EFIN', 'EETT', 'EVRR'].png`. The figure is reproduced.

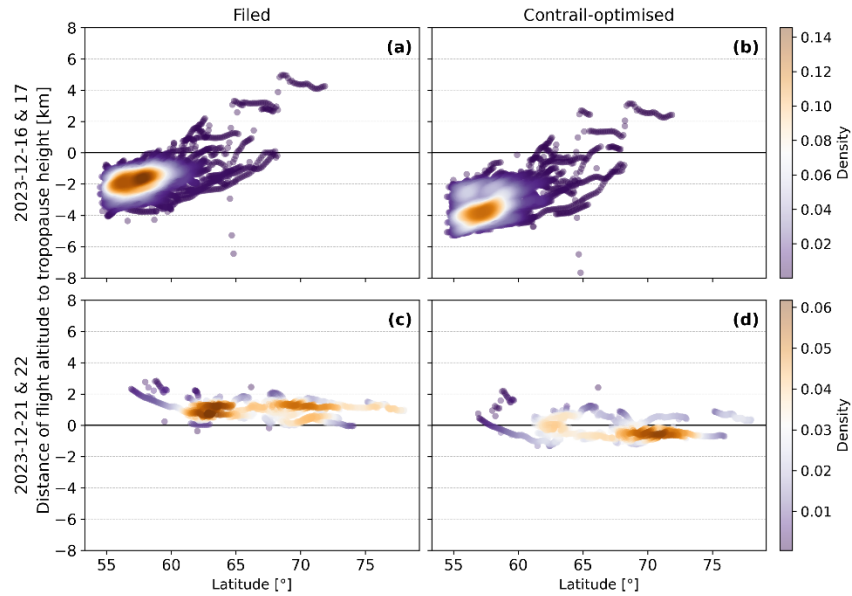


Figure 7:

The figure is created by `./notebooks/figure-07.ipynb` and saved to `./figures/fig07.png`. The figure is reproduced.

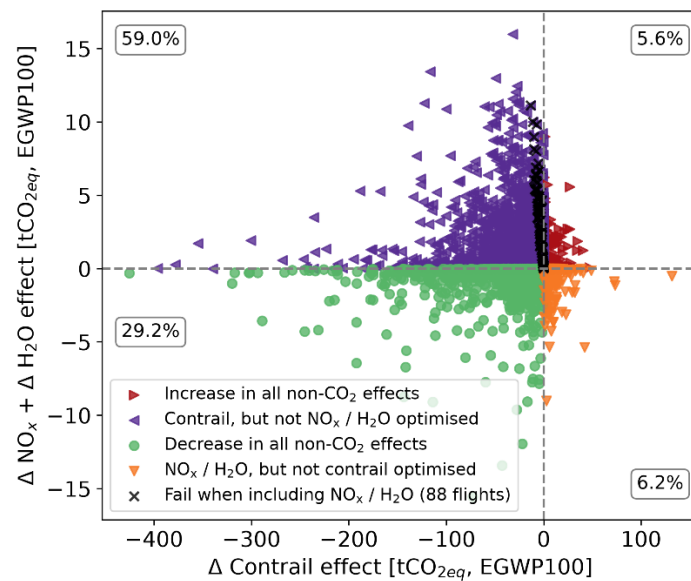


Figure 8:

The figure is created by ``./notebooks/figure-08.ipynb`` and saved to ``./figures/fig08.png``.
The figure is reproduced.

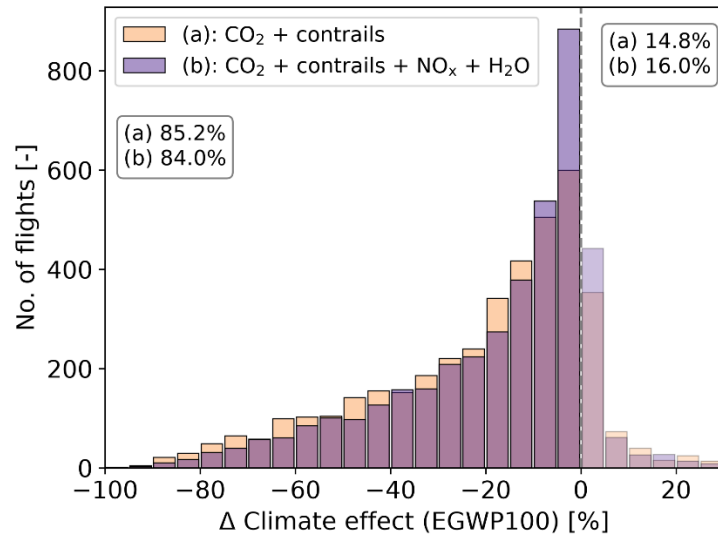


Figure 9:

The figure is created by ``./notebooks/figure-09.ipynb`` and saved to ``./figures/fig09.png``.
The figure requires ``sankey.py``. The codechecker followed the instructions specified in the ``README.md`` to download it from the original repo and comment lines 123 to 126. The figure is reproduced.

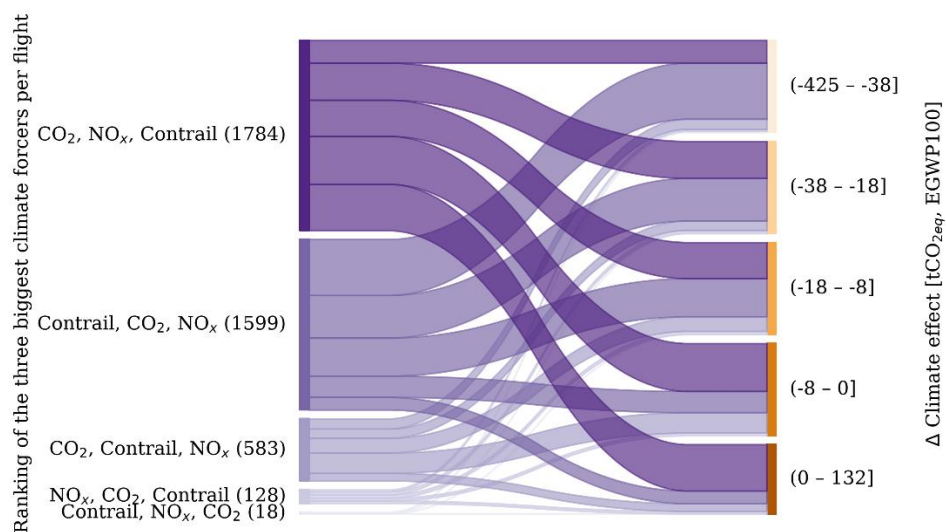


Figure 10:

The figure is created by `./notebooks/figure-10.ipynb` and saved to `./figures/fig10.png`.
The figure is reproduced.

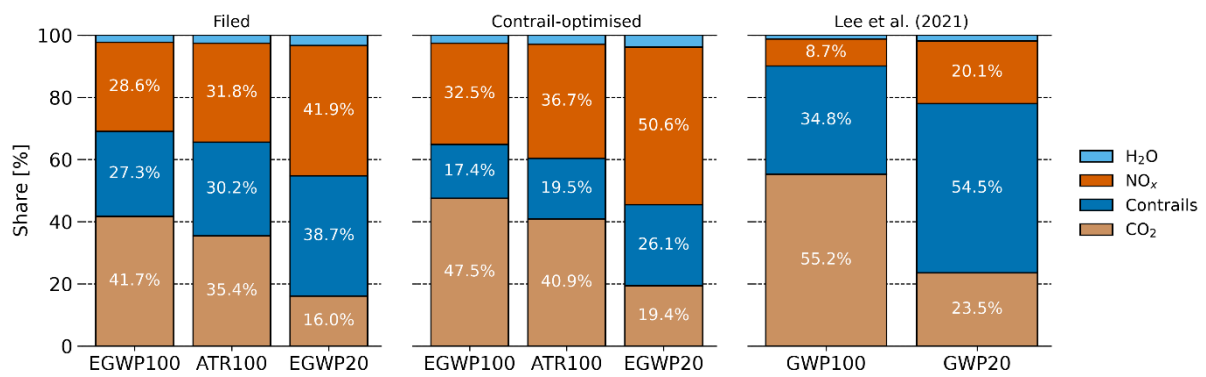


Figure 11:

The figure is created by `./notebooks/figure-11.ipynb` and saved to `./figures/fig11.png`.
The figure is reproduced.

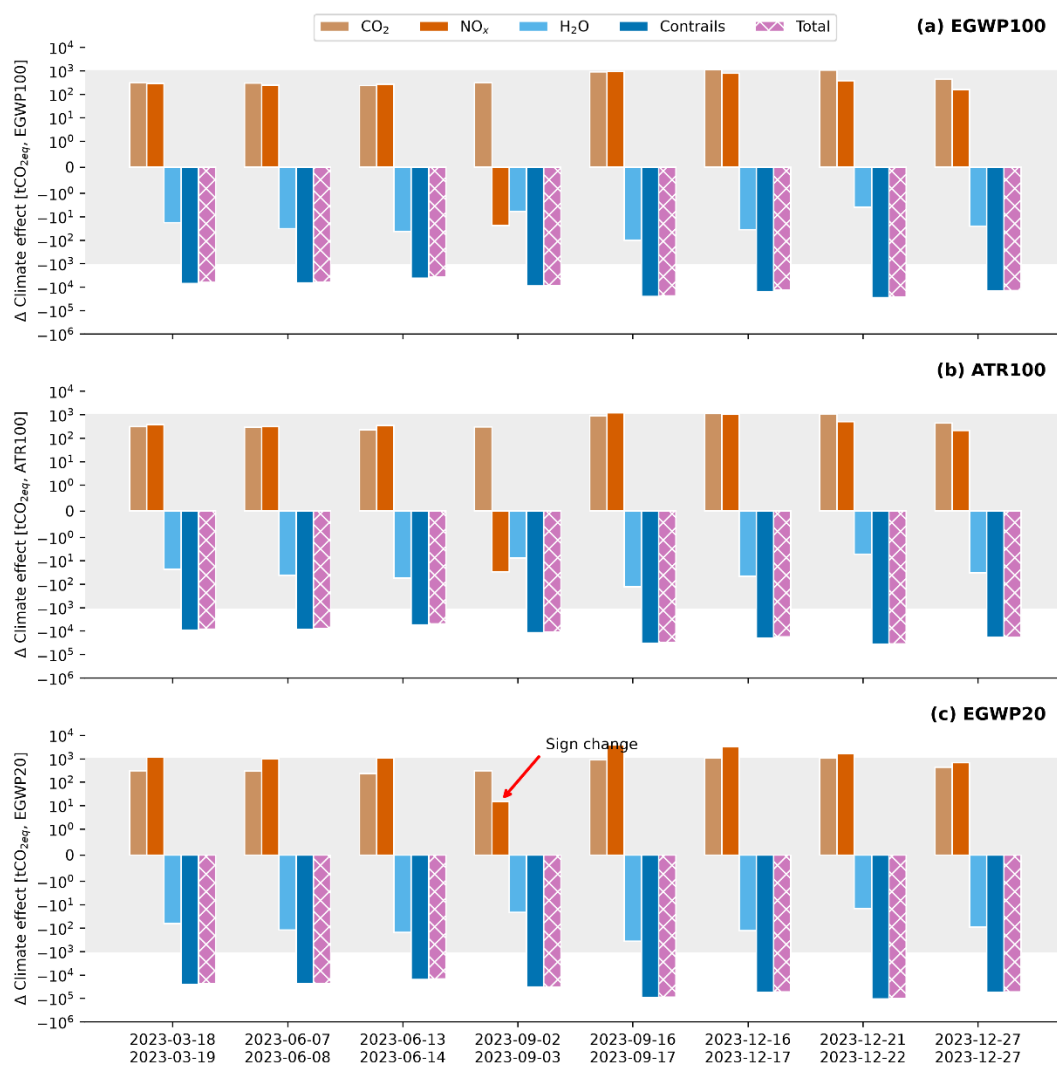


Figure C1:

The figure is created by `./notebooks/figure-C1.ipynb` and saved to `./figures/figC1.png`.
The figure is reproduced.

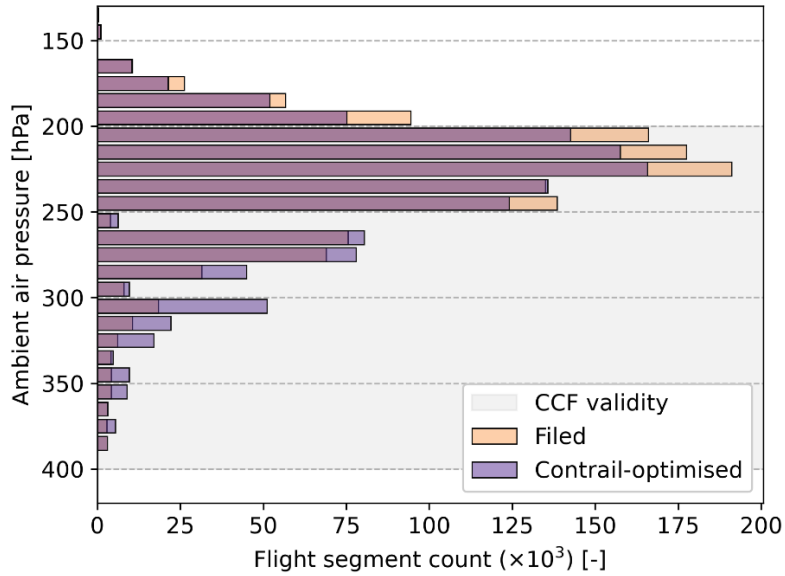


Figure C2:

The figure is created by `./notebooks/figure-C2.ipynb` and saved to `./figures/figC2.png`.
The figure is reproduced.

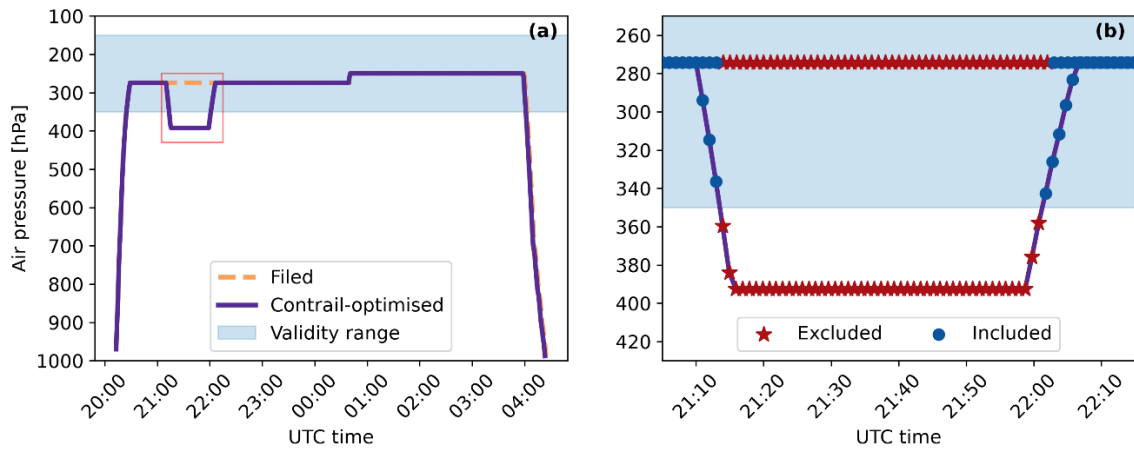


Figure C3:

The figure is created by ``./notebooks/figure-C3.ipynb`` and saved to ``./figures/figC3.png``.
The figure is reproduced.

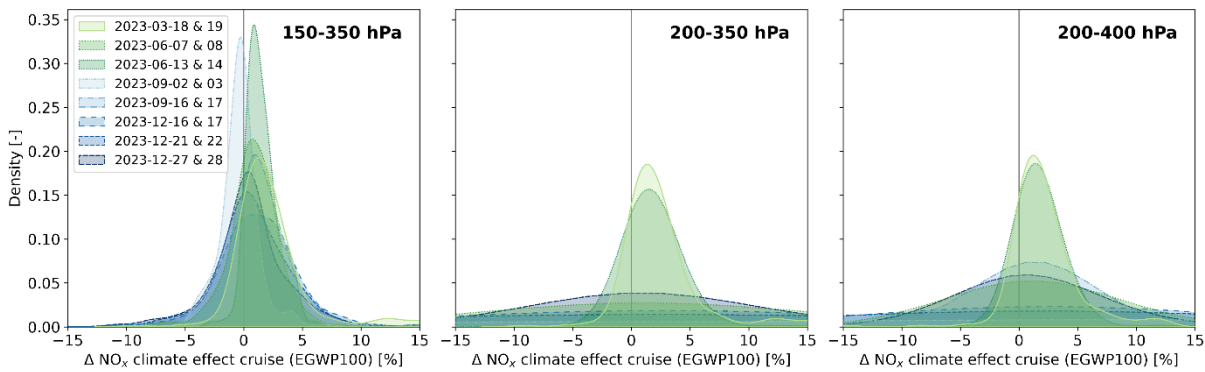


Figure D1:

The figure is created by ``./notebooks/figure-D1.ipynb`` and saved to ``./figures/figD1.png``.
The figure is reproduced.

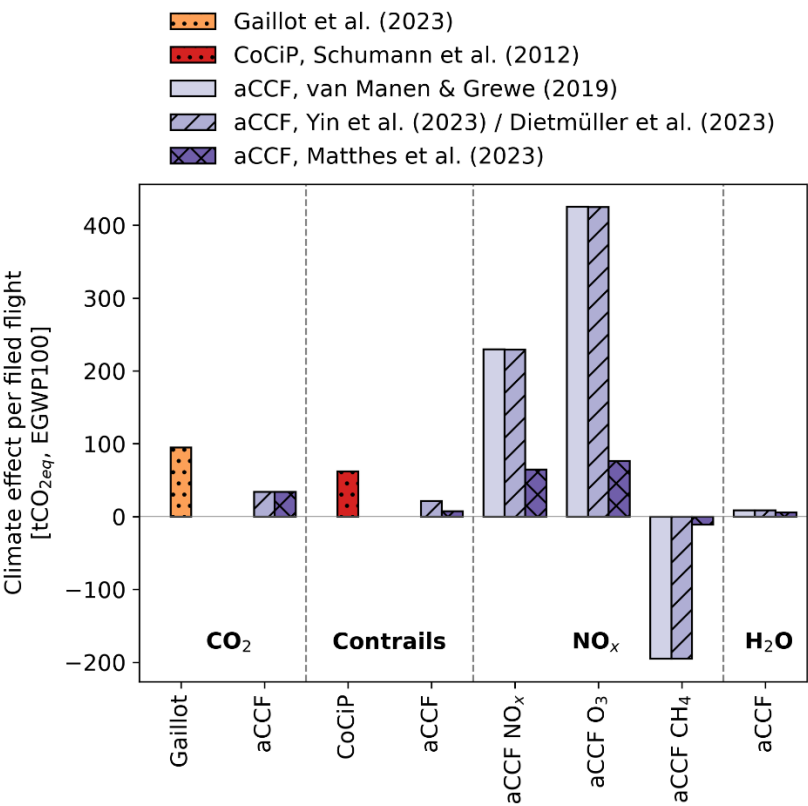
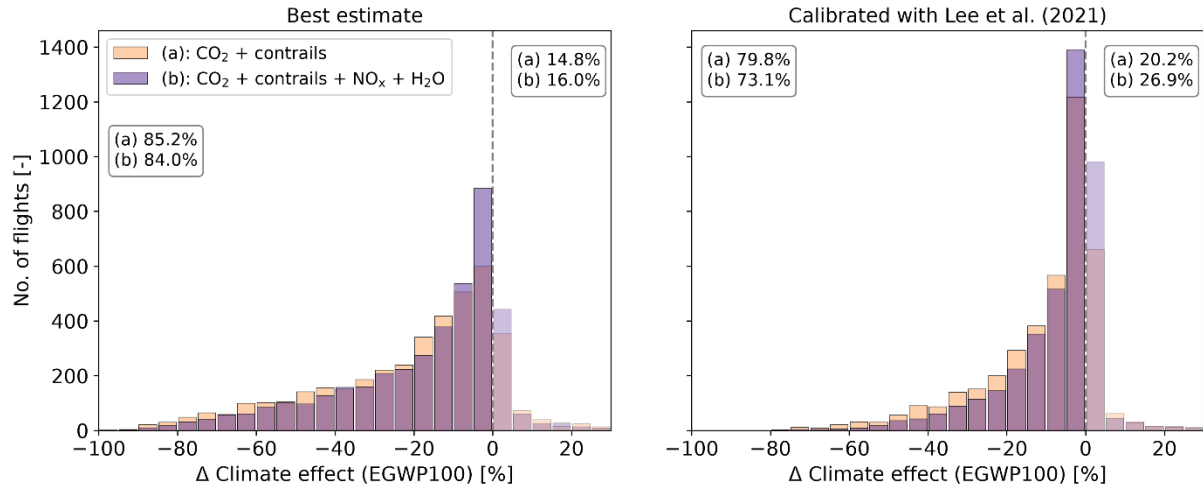


Figure E1:

The figure is created by `./notebooks/figure-E1.ipynb` and saved to `./figures/figE1.png`.
The figure is reproduced.



Acknowledgements

This CODECHECK was done as part of the Reproducibility Check initiative led by TU Delft's [Digital Competence Centre](#) and [4TU.ResearchData](#), in collaboration with the TU Delft [Faculty Data Stewards](#).

Citing this document

Andrews Mancilla, H. (2026). CODECHECK Certificate 2026-022. Zenodo.

<https://doi.org/10.5281/zenodo.21932994>

About CODECHECK

This certificate confirms that the codecheckers could independently reproduce the results of a computational analysis given the data and code from a third party. A CODECHECK does not check whether the original computation analysis is correct. However, as all materials required for the reproduction are freely available by following the links in this document, the reader can then study for themselves the code and data.